SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

TIRUCHIRAPPALLI CAMPUS

CONTINUOUS LEARNING ASSESSMENT – 2

CODE TRACE CHALLENGE

THINK ● TRACE ● PREDICT ● VERIFY

TASK – 2

Stack Operations Using an Array (C Programming)

DETAIL	INFORMATION
Submitted by	Navin N
Register Number	RA2510042050010
Programme / Department	B.Tech. – Computer Science and Business Systems (CSBS)
Course	21CSC201J – Data Structures and Algorithms
Assessment	Continuous Learning Assessment – 2 (Task – 2)
Academic Year	2026–2027

Department of Computer Science and Business System

DECLARATION OF ORIGINALITY

I, Navin N (Register Number: RA2510042050010), declare that this report presents my work for Continuous Learning Assessment – 2, Task 2, on stack operations using an array in C. The report records the supplied program's logic, manual trace, predicted output, analysis and handwritten evidence. I will verify the program by compiling and executing it before final submission and will record the actual results accurately.

Student Name: Navin N

Register Number: RA2510042050010

Signature: _____

Date: _____

CONTENTS

SECTION	TITLE
1	Aim and objectives
2	Task 1 – Code understanding
3	Task 2 – Manual code tracing
4	Task 3 – Value tracing and function return values
5	Task 4 – Output prediction and final state
6	Task 5 – What-if analysis
7	Task 6 – Error analysis and limitations
8	Task 7 – Verification after manual prediction
9	Additional test cases
10	Source code
11	Handwritten-work evidence
12	Conclusion
	References

1. Aim and objectives

Aim: To understand and manually trace the implementation of a stack using an array in C, predict its output and final state, and verify the prediction by executing the program.

Objectives

- Identify the stack data structure, variables, initial values, functions, conditions and loop.
- Trace each push, pop and display operation and record top, stack contents and outputs.
- Predict overflow, underflow, function return values and the final stack state.
- Perform a modified-operation analysis and identify limitations or possible improvements.
- Verify the original program and additional test cases using a C compiler.

2. Task 1 – Code understanding

2.1 Data structure used

The program implements a stack using a one-dimensional integer array. A stack follows the LIFO principle (Last In, First Out): the last value inserted is the first value removed. The variable top stores the index of the current top element.

2.2 Variables and initial values

Variable	Type / declaration	Initial value	Purpose
stack	int stack[SIZE]	Zero-initialized global array	Stores up to five stack values.
SIZE	#define SIZE 5	5	Defines the maximum capacity.
top	int	-1	Index of top element; -1 means empty.
item (push)	int parameter	Passed by caller	Value to insert into the stack.
item (pop)	local int	Assigned stack[top]	Temporarily stores the removed value.
item (main)	local int	Uninitialized until assigned	Receives the return value of pop().
i	int in display()	Initialized by for-loop	Traverses stack elements.

2.3 Functions and purpose

Function	Purpose	Return / effect
push(int item)	Checks whether stack is full. If not, increments top and stores item.	void; prints pushed value or overflow message.
pop()	Checks whether stack is empty. If not, saves stack[top], decrements top and returns saved value.	Returns removed integer; returns -1 on underflow.
display()	Prints an empty message or traverses indices 0 through top.	void; does not modify the stack.
main()	Calls push, pop and display in the specified order.	Returns 0 when program ends normally.

2.4 Conditions and loop

- Overflow condition: top == SIZE - 1. With SIZE = 5, overflow occurs when top == 4.
- Underflow condition: top == -1. A pop on an empty stack prints Stack Underflow and returns -1.
- Display loop: for (i = 0; i <= top; i++). It prints from index 0 to the top element.
- The program has no user-input statement; all operations are hard-coded in main().

3. Task 2 – Manual code tracing

The stack contents below show only the logical stack, from bottom to top. Array cells beyond top may still contain old values, but those cells are not part of the active stack.

Step	Operation	Item	top	Logical stack	Output
0	Initial state	—	-1	Empty	—
1	push(10)	10	0	10	Pushed: 10
2	push(20)	20	1	10, 20	Pushed: 20
3	push(30)	30	2	10, 20, 30	Pushed: 30
4	display()	—	2	10, 20, 30	Stack: 10 20 30
5	item = pop()	30	1	10, 20	Popped: 30
6	push(40)	40	2	10, 20, 40	Pushed: 40
7	push(50)	50	3	10, 20, 40, 50	Pushed: 50
8	push(60)	60	4	10, 20, 40, 50, 60	Pushed: 60
9	push(70)	70	4	10, 20, 40, 50, 60	Stack Overflow
10	display()	—	4	10, 20, 40, 50, 60	Stack: 10 20 40 50 60
11	pop()	60	3	10, 20, 40, 50	Popped: 60
12	pop()	50	2	10, 20, 40	Popped: 50
13	pop()	40	1	10, 20	Popped: 40
14	pop()	20	0	10	Popped: 20
15	pop()	10	-1	Empty	Popped: 10
16	display()	—	-1	Empty	Stack is Empty
17	pop()	-1	-1	Empty	Stack Underflow; Popped: -1

3.1 Display loop variable i

Display call	top at entry	Values of i	First i that stops loop	Result
1st display	2	0, 1, 2	3 (3 <= 2 is false)	Prints 10 20 30
2nd display	4	0, 1, 2, 3, 4	5 (5 <= 4 is false)	Prints 10 20 40 50 60
3rd display	-1	Loop not entered	Not applicable	Prints Stack is Empty

In the empty case, the if condition is true, so display() prints Stack is Empty and skips the for-loop. The local variable i is not used in that branch.

4. Task 3 – Value tracing and function return values

Call from main()	Value / state and function action	Return to main()	Value in main item
push(10), push(20), push(30)	Each value is inserted; top becomes 0, 1 and then 2.	No return (void)	Not applicable
pop() #1	Saves 30; top becomes 1.	30	30
push(40), push(50), push(60)	Values inserted; top becomes 2, 3 and then 4.	No return (void)	Not applicable
push(70)	Full check succeeds; no insertion.	No return (void)	Not applicable
pop() #2–#6	Removes 60, 50, 40, 20 and 10.	60, 50, 40, 20, 10	Updated each call
pop() #7	Prints underflow; returns -1.	-1	-1

The local variable item inside pop() is different from item in main(). The local variable stores the value being removed. The returned value is then assigned to main()'s item variable.

5. Task 4 – Output prediction and final state

Predicted complete output (before compiling):

Pushed: 10
Pushed: 20
Pushed: 30
Stack: 10 20 30
Popped: 30
Pushed: 40
Pushed: 50
Pushed: 60
Stack Overflow
Stack: 10 20 40 50 60
Popped: 60
Popped: 50
Popped: 40
Popped: 20
Popped: 10
Stack is Empty
Stack Underflow
Popped: -1

5.1 Predicted final state

Item	Predicted value
top	-1
Logical stack contents	Empty
Last value assigned to main()'s item	-1
Overflow occurrences	1 (push(70))
Underflow occurrences	1 (final pop())
Successful push operations	6 (10, 20, 30, 40, 50, 60)
Successful pop operations	6 (30, 60, 50, 40, 20, 10)

The array stores five elements. After the first three pushes and one pop, two elements remain. Three further pushes fill the array. The next push(70) is rejected because top is already 4. Five subsequent pops remove all active elements. The final pop is attempted when top is -1, causing underflow.

6. Task 5 – What-if analysis

Modification selected: Change push(60) in main() to push(65). All other statements remain unchanged.

Stage	Original program	Modified program
Input operation	push(60)	push(65)
After insertion	top=4; stack=10,20,40,50,60	top=4; stack=10,20,40,50,65
Next operation	push(70) → Overflow	push(70) → Overflow
Second display	Stack: 10 20 40 50 60	Stack: 10 20 40 50 65
Next pop	Returns 60	Returns 65
Remaining pops	50, 40, 20, 10	50, 40, 20, 10
Final state	top=-1; Empty	top=-1; Empty

Only the inserted value changes. The stack size and operation order remain the same, so overflow and underflow behavior do not change. The modified value 65 appears in the second display and is returned by the first pop after that display.

7. Task 6 – Error analysis and limitations

No incorrect stack-boundary condition is present in the supplied program. The overflow check top == SIZE - 1 and underflow check top == -1 are correct for this array implementation.

Observation	Explanation	Suggested improvement
Underflow sentinel ambiguity	pop() returns -1 on underflow, but -1 could also be a valid stored stack value. The caller cannot distinguish the two cases using the return value alone.	Return a status separately, use an output parameter, or use a sentinel disallowed by the input specification.
No user input	All values and operations are hard-coded in main().	For an interactive version, read menu choices and values from the user.
Display direction	display() prints from index 0 to top (bottom to top). This is valid, but it does not show the top element first.	If top-to-bottom display is desired, loop from top down to 0.
No explicit array clearing	pop() decrements top but does not erase the old array cell. The old value remains physically stored but is outside the logical stack.	Clearing stack[top] before decrement is optional and not required for correctness in this integer example.

The program's global array is zero-initialized by C, but the logical stack is empty because top starts at -1. Values physically left in stack[] after all pops do not mean the stack still contains elements.

8. Task 7 – Verification after manual prediction

Verification status: Completed. The program was compiled successfully with GCC and executed. The actual output matched the predicted output, including the final top value, empty logical stack, one overflow and one underflow.

Comparison item	Predicted result	Actual result – fill after running	Match / notes
Complete output	As listed in Section 5	Matched the complete GCC output	Match
Final top	-1	-1	Match
Final logical stack	Empty	Empty	Match
Overflow	Once at push(70)	Once at push(70)	Match
Underflow	Once at final pop()	Once at final pop()	Match

9. Additional test cases

The following are separate operation sequences for testing the push/pop functions. Each test was executed independently, starting with top = -1 and an empty stack. The actual results are recorded in the final column and the corresponding execution screenshots are included in Section 10.1.

Test	Operations	Expected stack / result	Actual result
1 – Basic LIFO	push(10), push(20), pop()	pop returns 20; top=0; stack=[10]	Returned 20; stack=[10]; top=0.
2 – Overflow	push(1), push(2), push(3), push(4), push(5), push(6)	First five insert; sixth push reports Stack Overflow; top=4	First five pushes succeeded; sixth caused Stack Overflow; stack=[1,2,3,4,5]; top=4.
3 – Underflow	pop() on an empty stack	Prints Stack Underflow; pop returns -1; top remains -1	Stack Underflow printed; returned -1; top=-1.
4 – Fill and empty	push(4), push(8), push(12), pop(), pop(), pop()	Returns 12, 8, 4; final top=-1	Popped 12, 8, 4; stack empty; top=-1.

The actual results matched the expected behavior for all four test cases.

10. Source code

```
#include <stdio.h>
#define SIZE 5

int stack[SIZE];
int top = -1;

void push(int item) {
   if (top == SIZE - 1) {
      printf("Stack Overflow\\n");
   } else {
      top++;
      stack[top] = item;
      printf("Pushed: %d\\n", item);
   }
}

int pop() {
   if (top == -1) {
      printf("Stack Underflow\\n");
      return -1;
   } else {
      int item = stack[top];
      top--;
      return item;
   }
}

void display() {
   int i;
   if (top == -1) {
      printf("Stack is Empty\\n");
   } else {
      printf("Stack: ");
      for (i = 0; i <= top; i++) {
         printf("%d ", stack[i]);
      }
      printf("\\n");
   }
```

```
}

int main() {
    int item;
    push(10);
    push(20);
    push(30);
    display();

    item = pop();
    printf("Popped: %d\\n", item);

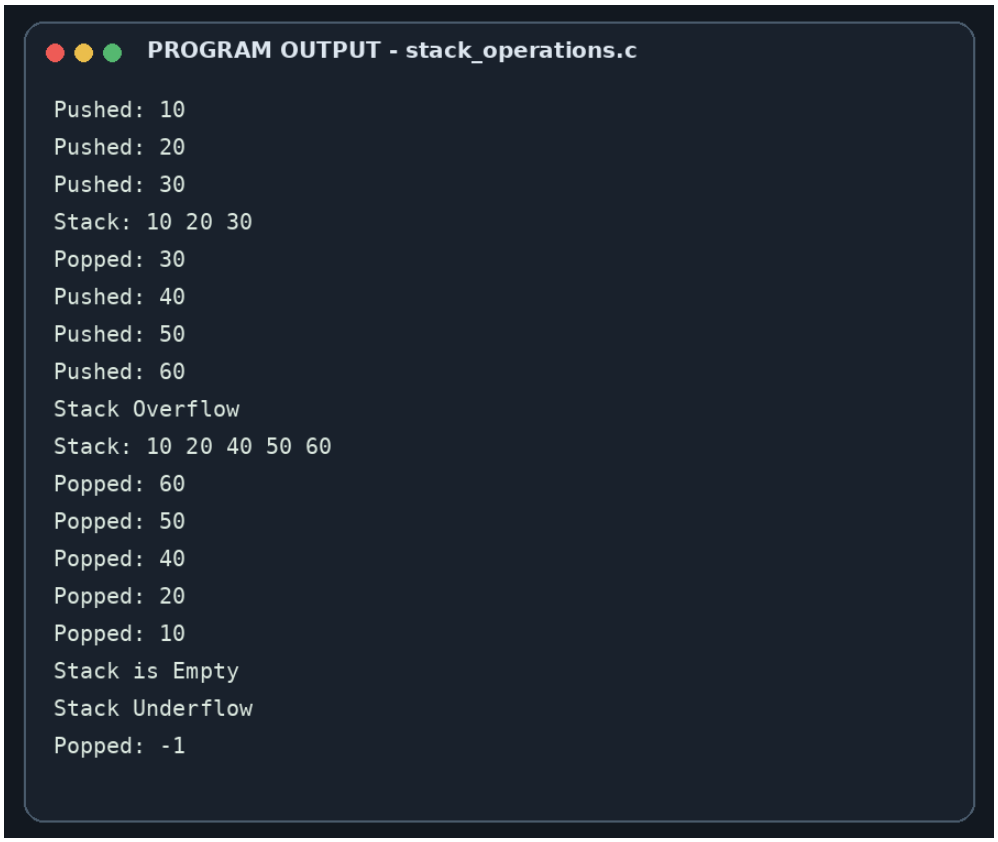
    push(40);
    push(50);
    push(60);
    push(70);
    display();

    item = pop(); printf("Popped: %d\\n", item);
    item = pop(); printf("Popped: %d\\n", item);
    item = pop(); printf("Popped: %d\\n", item);
    item = pop(); printf("Popped: %d\\n", item);
    item = pop(); printf("Popped: %d\\n", item);

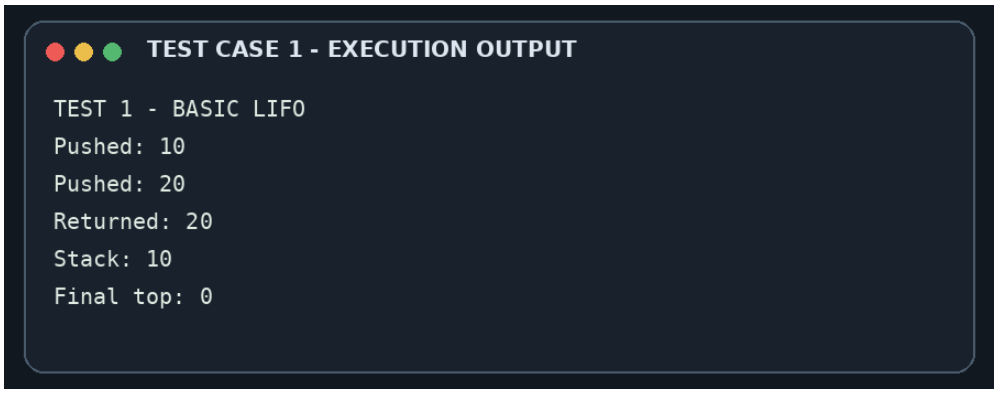
    display();
    item = pop();
    printf("Popped: %d\\n", item);
    return 0;
}
```

10.1 Actual program execution output

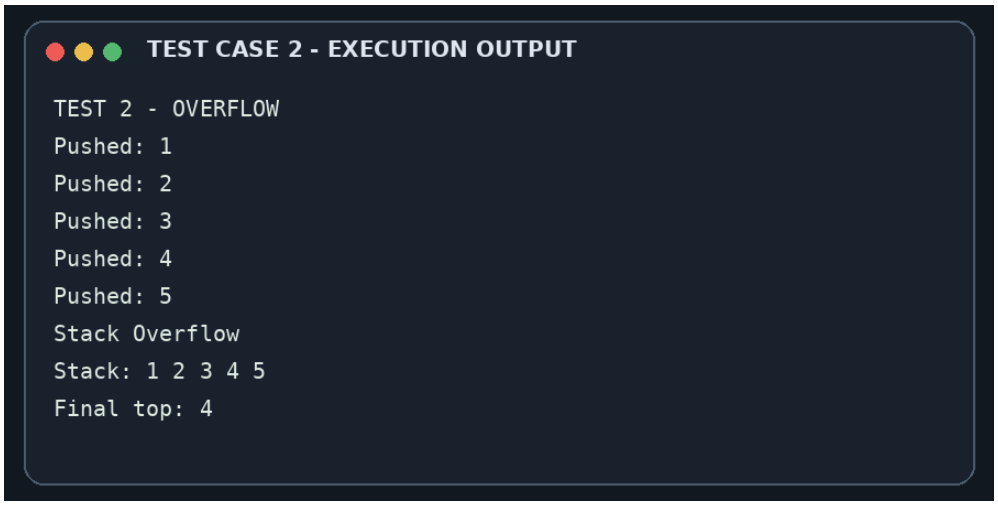
The following screenshots were generated from successful GCC compilations and executions of the supplied program and each independent test sequence.



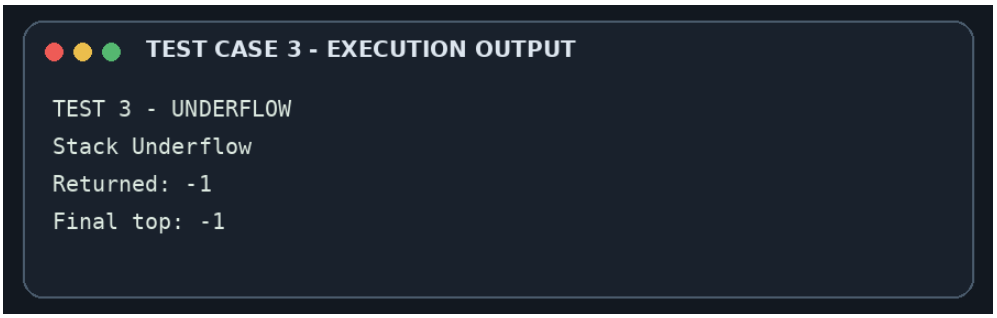
TEST CASE 1 – BASIC LIFO



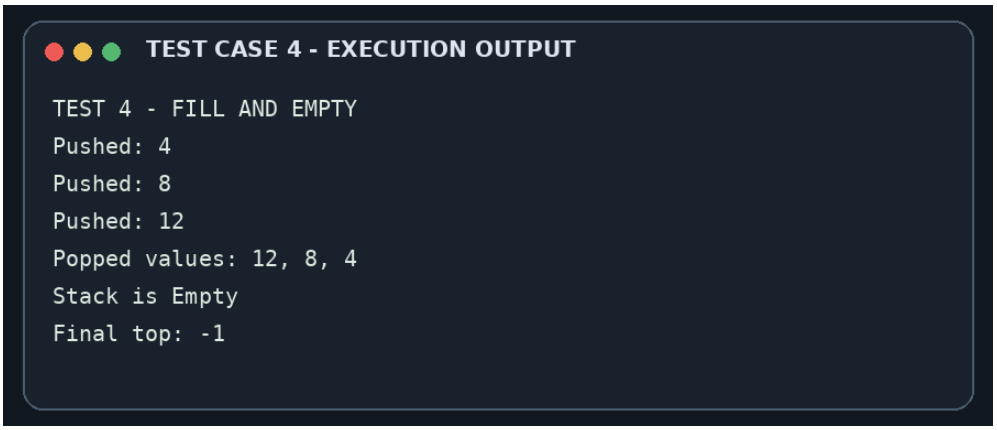
TEST CASE 2 – OVERFLOW



TEST CASE 3 – UNDERFLOW



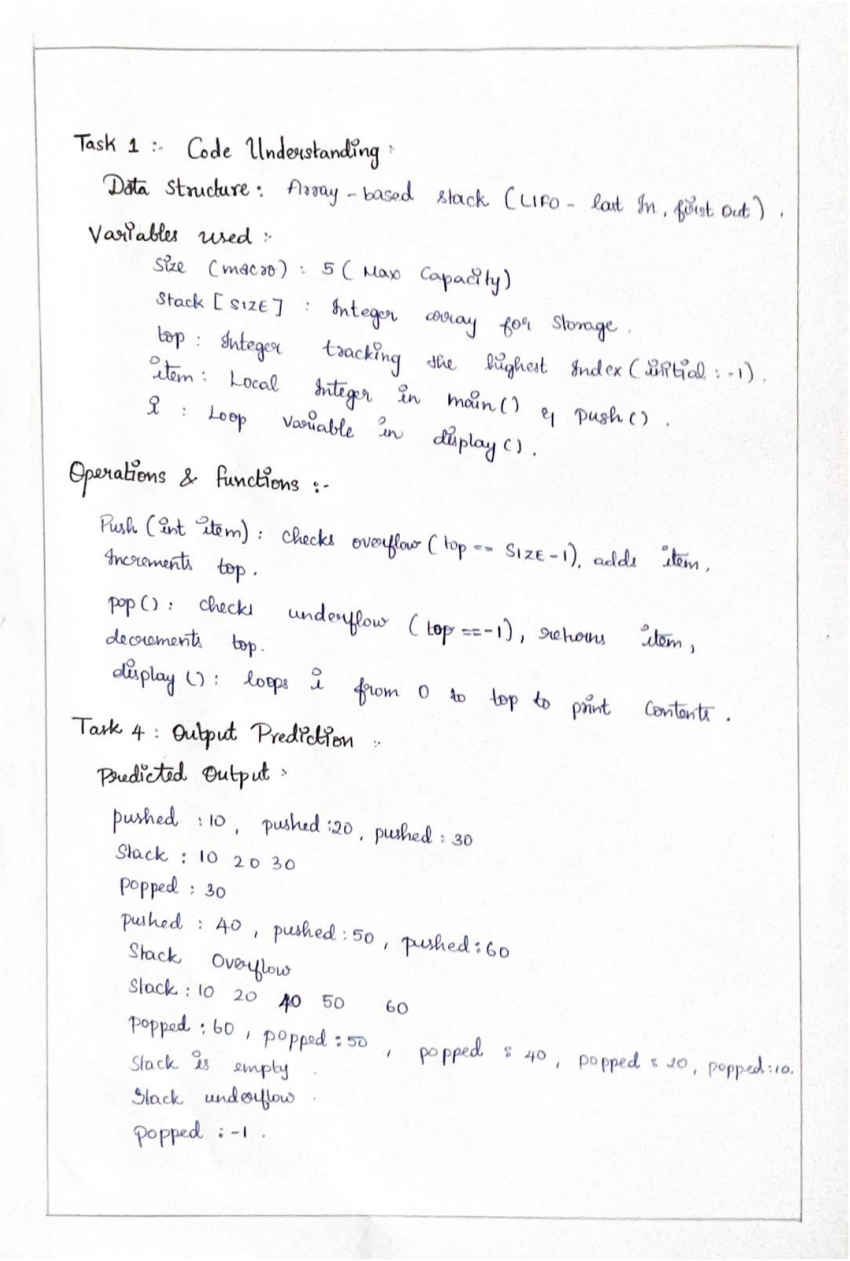
TEST CASE 4 – FILL AND EMPTY



11. Handwritten-work evidence

The following pages are inserted from the separately supplied DSA_Task2.pdf. The handwritten pages are retained as supplied.

Handwritten evidence – Page 1



Task 1 :- Code Understanding :

Data Structure : Array - based stack (LIFO - Last In, First Out).

Variables used :

Size (macro) : 5 (Max Capacity)

Stack [SIZE] : Integer array for storage.

top : Integer tracking the highest index (initial : -1).

item : Local integer in main() & push().

i : Loop variable in display().

Operations & functions :-

Push (int item) : checks overflow (top == SIZE-1), adds item, increments top.

pop() : checks underflow (top == -1), returns item, decrements top.

display() : loops i from 0 to top to print contents.

Task 4 : Output Prediction :

Predicted Output :

pushed : 10, pushed : 20, pushed : 30

Stack : 10 20 30

popped : 30

pushed : 40, pushed : 50, pushed : 60

Stack Overflow

Stack : 10 20 40 50 60

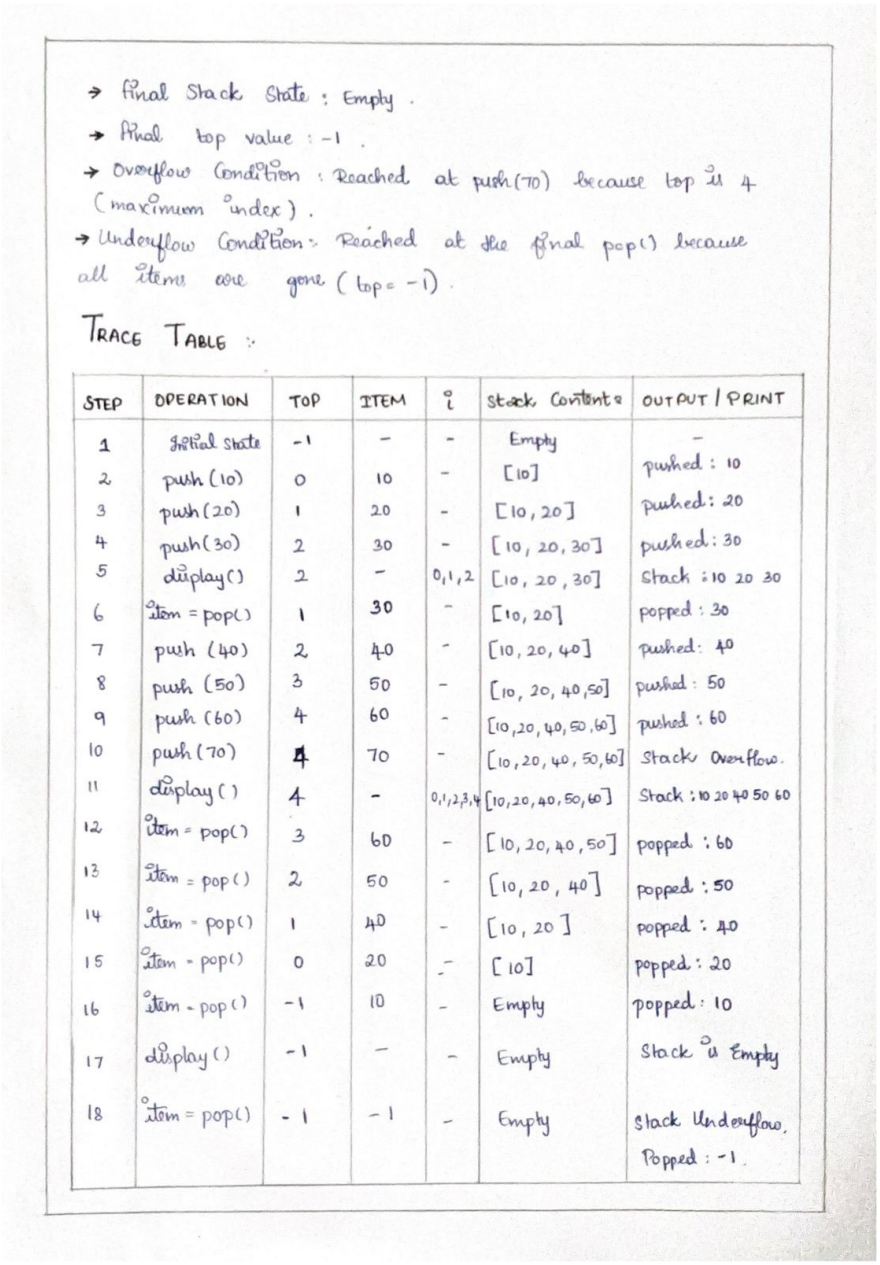
popped : 60, popped : 50, popped : 40, popped : 20, popped : 10.

Stack is empty.

Stack underflow.

popped : -1.

Handwritten evidence – Page 2



→ Final Stack State : Empty.

→ Final top value : -1.

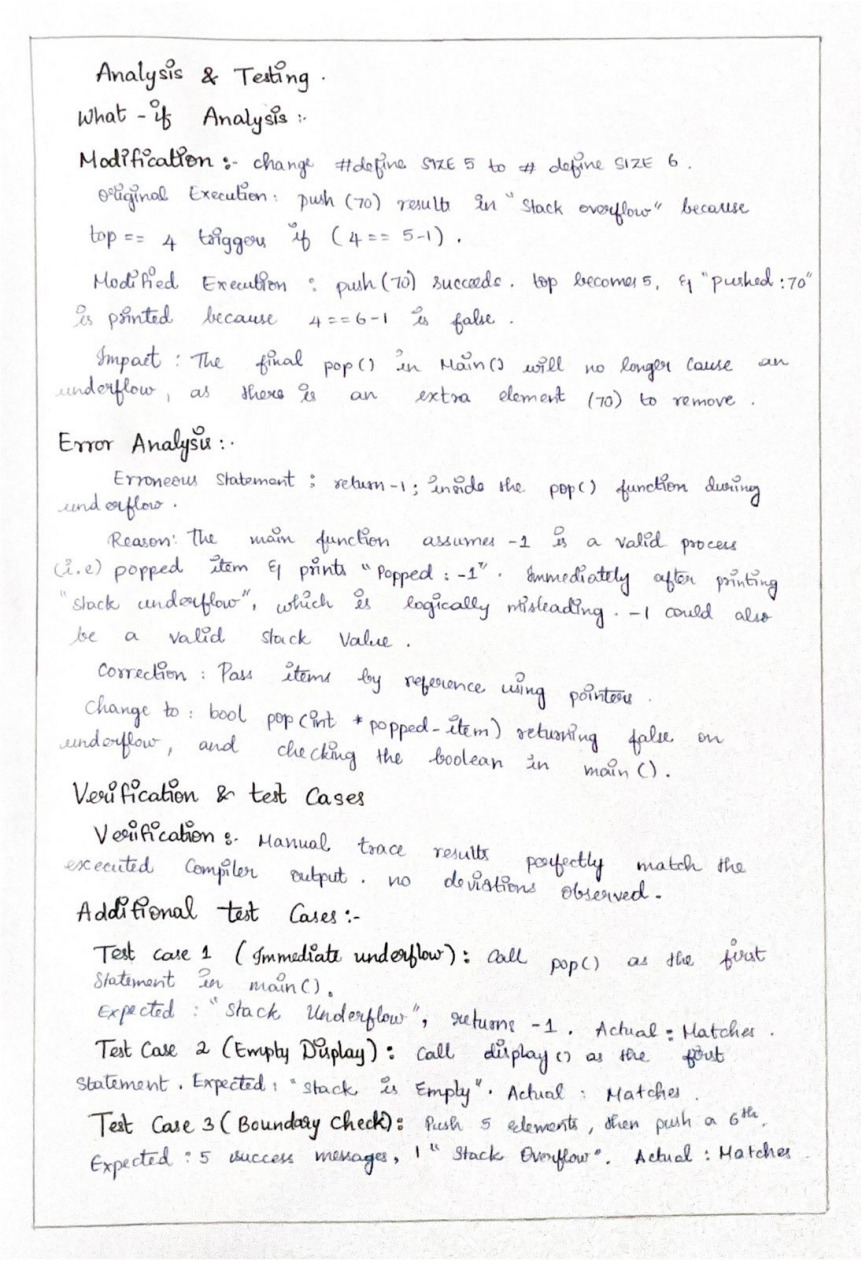
→ Overflow Condition : Reached at push(70) because top is 4 (maximum index).

→ Underflow Condition : Reached at the final pop() because all items are gone (top = -1).

TRACE TABLE :

STEP	OPERATION	TOP	ITEM	i	Stack Contents	OUTPUT / PRINT
1	Initial state	-1	–	–	Empty	–
2	push(10)	0	10	–	[10]	pushed : 10
3	push(20)	1	20	–	[10, 20]	pushed: 20
4	push(30)	2	30	–	[10, 20, 30]	pushed: 30
5	display()	2	–	0,1,2	[10, 20, 30]	Stack : 10 20 30
6	item = pop()	1	30	–	[10, 20]	popped : 30
7	push (40)	2	40	–	[10, 20, 40]	pushed: 40
8	push (50)	3	50	–	[10, 20, 40, 50]	pushed : 50
9	push (60)	4	60	–	[10, 20, 40, 50, 60]	pushed : 60
10	push (70)	4	70	–	[10, 20, 40, 50, 60]	Stack Overflow.
11	display()	4	–	0,1,2,3,4	[10, 20, 40, 50, 60]	Stack : 10 20 40 50 60
12	item = pop()	3	60	–	[10, 20, 40, 50]	popped : 60
13	item = pop()	2	50	–	[10, 20, 40]	popped : 50
14	item = pop()	1	40	–	[10, 20]	popped : 40
15	item = pop()	0	20	–	[10]	popped : 20
16	item = pop()	-1	10	–	Empty	popped : 10
17	display()	-1	–	–	Empty	Stack is Empty
18	item = pop()	-1	-1	–	Empty	Stack Underflow, Popped : -1.

Handwritten evidence – Page 3



Analysis & Testing.

What - if Analysis :-

Modification :- change #define SIZE 5 to # define SIZE 6.

Original Execution: push (70) results in "Stack overflow" because top == 4 triggers if (4 == 5-1).

Modified Execution : push (70) succeeds. top becomes 5. & "pushed : 70" is printed because 4 == 6-1 is false.

Impact : The final pop() in Main() will no longer cause an underflow, as there is an extra element (70) to remove.

Error Analysis :-

Erroneous Statement : return -1; inside the pop() function during underflow.

Reason: the main function assumes -1 is a valid process (i.e) popped item & prints "Popped : -1". Immediately after printing "Stack underflow", which is logically misleading. -1 could also be a valid stack Value.

Correction : Pass items by reference using pointers.

Change to : bool pop(int *popped_item) returning false on underflow, and checking the boolean in main().

Verification & test Cases

Verification :- Manual trace results perfectly match the executed Compiler output. no deviations Observed.

Additional test Cases :-

Test case 1 (Immediate underflow): Call pop() as the first Statement in main().
Expected : "Stack Underflow", returns -1. Actual : Matches.

Test Case 2 (Empty Display): Call display() as the first statement. Expected : "Stack is Empty". Actual : Matches.

Test Case 3 (Boundary Check): Push 5 elements, then push a 6th.
Expected : 5 success messages, 1 "Stack Overflow". Actual : Matches

12. Conclusion

This assessment demonstrates the working of a stack implemented using an array and the LIFO principle. The top variable identifies the most recently inserted active element. The push operation inserts an item when capacity is available, while pop removes and returns the top item. The supplied program reaches overflow when push(70) is attempted on a full stack and reaches underflow when the final pop is attempted on an empty stack. The predicted final logical state is an empty stack with top = -1. The prediction must be confirmed by running the program and recording the actual output.

References

1. Supplied Task 2 stack-operations report used as the content reference.

2. DSA_Task2.pdf, three handwritten assessment pages supplied for evidence.